Docker Cloud

Cloud de fichiers éphémères : chaque fichier a une durée de vie, un worker supprime les fichiers expirés

M2 – Cours Docker · Thomas Abadie · Octobre 2026

Dépôt du projet : **github.com/AbadieThomasPro/M2_TP_Docker**

Branche `main` · lancement : `docker compose up -d --build --wait` puis `http://127.0.0.1:8080`

5	0	1	0	80 %
images construites depuis `alpine`	image applicative du Docker Hub	seul port publié (front, 8080)	code de sortie de chaque service à l'arrêt	plafond machine de l'autoscaling

Rendu attendu → où le trouver

Attendu (sujet)	Dans le dépôt
Lien git	Ci-dessus, branche `main`
Plusieurs images	`Frontend/` `Gateway/` `Backend/` `Cleaner/` `Autoscaler/`
Un ou plusieurs compose	`docker-compose.yml` (principal), `docker-stack.yml` (Swarm)
Documentation des choix	Ce document (synthèse) et le dossier `docs/` (détail, mesures) ; `README.md` : installation
Schéma de l'architecture	Pages 2, 3 et 7 (avec légendes)

Barème → page

Critère	Page
Dépendances installées	4
Manipulations sur l'OS	4
Arguments attendus	5
Entrypoints choisis	5
Arguments traduits dans le compose	5
Limitation des ressources expliquée	6
SIGTERM gérés	6
Dépendances entre conteneurs	3
Schéma des communications	2, 3, 7

Les 5 images en une ligne

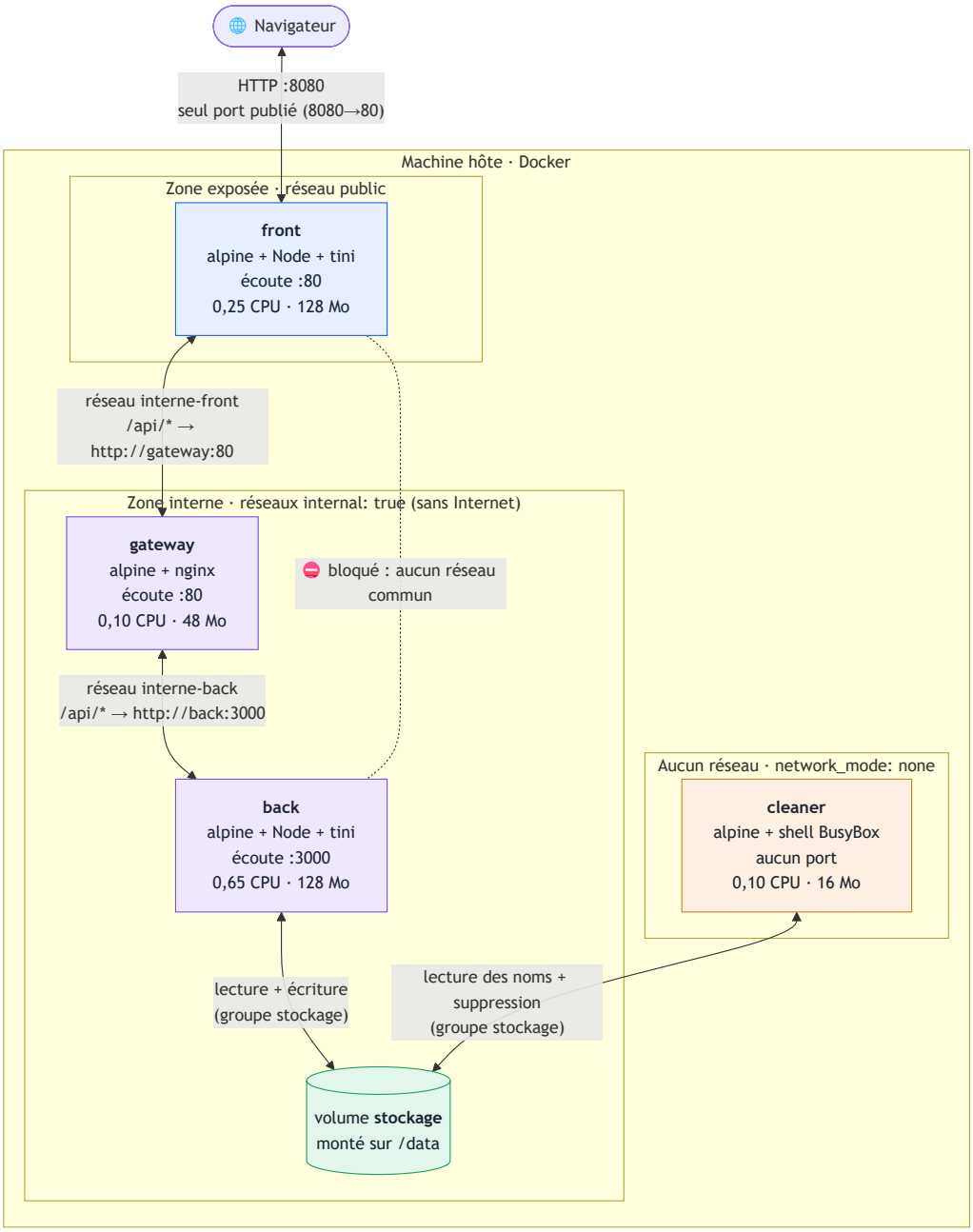
Image	Rôle	Pourquoi elle existe
front	Sert l'application Angular, relaie `/api`	Seul point d'entrée : le navigateur ne connaît qu'une adresse
gateway (serveur web)	nginx, passerelle d'API	Seul chemin vers le back : filtre les routes, refuse les envois trop gros (413)
back	API Express : envoyer, lister, télécharger, supprimer	Sans état (expiration dans le nom du fichier) : peut être multiplié
cleaner	Worker : supprime les fichiers expirés	Aucun réseau : ne parle à personne, agit sur le volume partagé
autoscaler (Swarm)	Ajuste le nombre d'instances selon CPU / mémoire	Swarm n'a pas d'autoscaling natif, et ceux du Hub sont interdits

Base `alpine:3.22` (autorisée par l'enseignant) : tout le reste est installé et configuré par nos soins.

Architecture (docker-compose)

Schéma des communications

Le livrable principal. Le front est le seul service publié ; la gateway est le seul pont vers le back.



Public : joignable depuis l'hôte **Interne** : réseaux `internal` , sans Internet **Données** : volume nommé

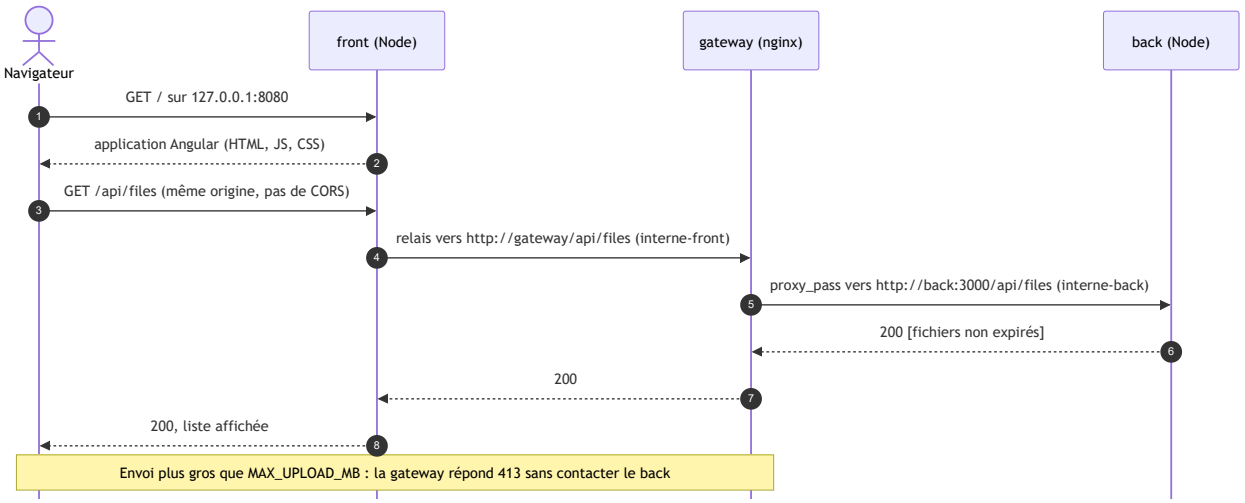
Isolé : aucune interface réseau échange (requête → réponse, ou lecture / écriture) chemin bloqué

Élément	Choix	Pourquoi
Port publié	`8080` → `80` du front uniquement	Une seule porte d'entrée ; 8080 évite le port 80 souvent pris sur l'hôte
Réseaux	`public` , `interne-front` , `interne-back`	Avec un seul réseau interne, le front joignait `back:3000` en direct (testé) ; séparés, il doit passer par la gateway
Volume	`stockage` sur `/data` (back + cleaner)	Volume nommé : géré par Docker, portable, survit à `down` et aux rebuilds
Droits partagés	Groupe `stockage` (GID 1500), dossier `2775`	Le cleaner supprime les fichiers du back sans être root ni être « back »

Communications et ordre de démarrage

Schéma des communications · Dépendances entre conteneurs

Chemin d'une requête : le navigateur ne connaît que le front, qui relaie vers la gateway, qui relaie vers le back.



requête ····· réponse ① à ⑧ : ordre des échanges

Ordre de démarrage (`depends_on: condition: service_healthy`)

1 · back healthy → 2 · gateway + 2 · cleaner healthy → 3 · front

Service	Attend	Healthcheck	Pourquoi cet ordre
back	–	`GET /health`	Premier : c'est lui qui initialise les droits du volume
gateway	back *healthy*	`GET /health` sur `127.0.0.1`	Sans back prêt, elle répondrait `502` à chaque appel (testé)
cleaner	back *healthy*	Date de la dernière passe (pas de réseau)	Le premier conteneur qui monte un volume neuf y copie **ses** droits : le back doit passer avant
front	gateway *healthy*	`GET /`	La page n'est servie qu'une fois toute la chaîne vers l'API prête

Healthchecks définis dans chaque Dockerfile ; en Swarm (pas de `depends_on`), l'image du cleaner contient aussi le dossier du volume : l'ordre n'a plus d'importance (vérifié).

Les images : dépendances et OS

Dépendances installées · Manipulations sur l'OS

Toutes partent de `alpine:${ALPINE_VERSION}` (3.22, version fixée dans `.env`, jamais `latest`) et installent elles-mêmes leurs outils.

Dépendances installées (`apk add --no-cache`)

Image	Paquets	Pourquoi	Taille
front	build : `nodejs npm` final : `nodejs tini tzdata`	Multi-stage : Angular est compilé dans un stage jeté ; l'image finale n'a ni npm ni sources. `tini` relaie SIGTERM à Node. `tzdata` : logs à l'heure de Paris	114 Mo
gateway	`nginx gettext-envsubst tzdata`	nginx : reverse proxy léger ; `gettext-envsubst` seul (pas tout gettext) pour générer la config au run	18 Mo
back	deps : `nodejs npm` → `npm ci --omit=dev` final : `nodejs tini tzdata`	Multi-stage : seul `node_modules` (express, multer) est copié, pas npm. `npm ci` : versions exactes du lock	120 Mo
cleaner	**aucun**	BusyBox fournit déjà `sh awk xargs rm find setsid` : pas de runtime pour une boucle de quelques lignes	13 Mo
autoscaler	`docker-cli`	Le client suffit (le moteur est celui de l'hôte) ; ni `bash` ni `curl`, inutiles	57 Mo

Manipulations sur l'OS

Manipulation	Où	Pourquoi
Utilisateur dédié, `USER` non-root	toutes	En cas de faille, l'attaquant n'est pas root ; le code reste à root (lecture seule pour l'utilisateur)
UID explicites : back `1001`, cleaner `1002`, autoscaler `1003`	back, cleaner, autoscaler	Par défaut chaque image donne 1000 : back et cleaner étaient confondus par le noyau (constaté en test)
Groupe `stockage` au GID `1500` (ARG `STORAGE_GID`)	back, cleaner	Seul le numéro compte pour le noyau : même valeur dans les deux images, fournie par `.env`
`mkdir /data/files/.incoming` + `chmod 2775` avant `VOLUME`	back, cleaner	Docker recopie ces droits dans le volume neuf ; `2775` = groupe en écriture + setgid (les fichiers héritent du groupe)
Ajout au groupe du socket Docker (ARG `DOCKER_GID`, 0 mesuré)	autoscaler	Piloter Swarm sans être root : groupe secondaire seulement
`COPY --chmod=755`, code copié en dernier	toutes	Une couche de moins ; le cache garde OS et dépendances quand seul le code change
`read_only: true` + `tmpfs /tmp` limité	toutes (au run)	Impossible de modifier le code ou de déposer un binaire ; seul le volume ou `/tmp` (en RAM) est inscriptible

Entrypoints et arguments

Entrypoints · Arguments attendus · Traduits dans le compose

Entrypoints choisis (toujours en forme exec)

Image	ENTRYPOINT / CMD	Pourquoi
front, back	`["/sbin/tini","--"]` + `["node","…"]`	Node en PID 1 ignore SIGTERM : `tini` le relaie et nettoie les zombies. Sans shell intermédiaire
gateway	`["/entrypoint.sh"]` + `["nginx","-g","daemon off;",…]`	Modèle « wrapper d'init » : le script génère la config depuis l'ENV puis `exec "$@"` → nginx devient PID 1. CMD remplaçable (`nginx -t`). `STOPSIGNAL SIGQUIT` : finit les requêtes en cours
cleaner, autoscaler	`["/usr/local/bin/…sh"]`	Script shell conçu pour être PID 1 : il gère lui-même SIGTERM (`trap`), pas besoin de tini

Arguments : ARG (build) → ENV (défaut de l'image) → `.env` (déploiement)

Niveau	Rôle	Traduction dans le compose
ARG	Valeur figée au build : `ALPINE_VERSION` , `PORT` (→ `EXPOSE`), `STORAGE_GID` , `DOCKER_GID`	`build.args` ← `.env`
ENV	Défaut dans l'image : l'image démarre seule, sans compose	`environment:` ← `.env` (changer une valeur = sans rebuild)
.env	Source unique de toutes les valeurs ; versionné (aucun secret)	Aucune valeur en dur dans le compose, sauf les noms de service

Arguments attendus au run (principaux)

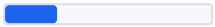
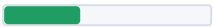
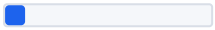
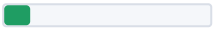
Variable	Image	Valeur (.env)	Pourquoi
`PORT`	toutes sauf cleaner	80 / 3000	Même valeur pour `EXPOSE` et l'écoute
`NODE_OPTIONS`	front, back	`--max-old-space-size=64`	Tas Node à la moitié de la limite mémoire : Node libère avant d'être tué (OOM)
`MAX_UPLOAD_MB`	gateway, back	50	Refus (413) dès la gateway, double sécurité dans le back
`TTL_DEFAULT_H` / `TTL_MAX_H`	back	24 / 168	Durée de vie par défaut (1 journée) et max (7 jours)
`STORAGE_QUOTA_MB`	back	1024	Protège le disque de l'hôte (`507` au-delà)
`WORKER_PROCESSES` / … `CONNECTIONS`	gateway	1 / 512	Un worker par CPU alloué (moins d'un CPU)
`BACK_HOST` , `GATEWAY_HOST`	gateway, front	`back` , `gateway`	Noms de service résolus par le DNS Docker, pas d'IP en dur
`CLEANUP_INTERVAL_S`	cleaner	60	L'API masque déjà les expirés : seule la libération du disque attend
`AUTOSCALE_*`	autoscaler	voir page 7	Seuils issus du benchmark
`cpus` / `memory`	tous	voir page 6	`deploy.resources` ← `.env`

Ressources et arrêt propre

Limitation des ressources · SIGTERM

Limites dimensionnées par un benchmark (outil `ab` construit pour l'occasion) : CPU proportionnel au coût mesuré par requête (back / front / gateway ≈ 7 / 3 / 1), mémoire selon le pic mesuré.

Limites par service (compose)

Service	CPU		Mémoire		Réserv.	Justification mesurée
back	0,65		128 Mo		32 Mo	Le plus coûteux : ~2,8 ms CPU par liste ; pic ~33 Mo (×4 de marge)
front	0,25		128 Mo		32 Mo	~1,2 ms par requête (relaie tout) ; pic ~37 Mo
gateway	0,10		48 Mo		8 Mo	~0,4 ms par requête ; nginx ~4 Mo, pic 26 Mo en envois + tmpfs de 8 Mo
cleaner	0,10		16 Mo		6 Mo	10 000 fichiers supprimés en 6 s ; pic ~4 Mo ; 0 % au repos

Réglage	Pourquoi
`memswap_limit` = limite mémoire	Sinon Docker double la limite en swap (mesuré) : le service ralentit au lieu d'être arrêté
Réservations mémoire (et CPU en Swarm)	Minimum garanti ; en Swarm, une instance n'est placée que si la machine peut la tenir
Test de dépassement	Back limité à 6 Mo : tué (`OOMKilled` , code 137) puis relancé par `restart: unless-stopped`

SIGTERM : arrêt propre mesuré

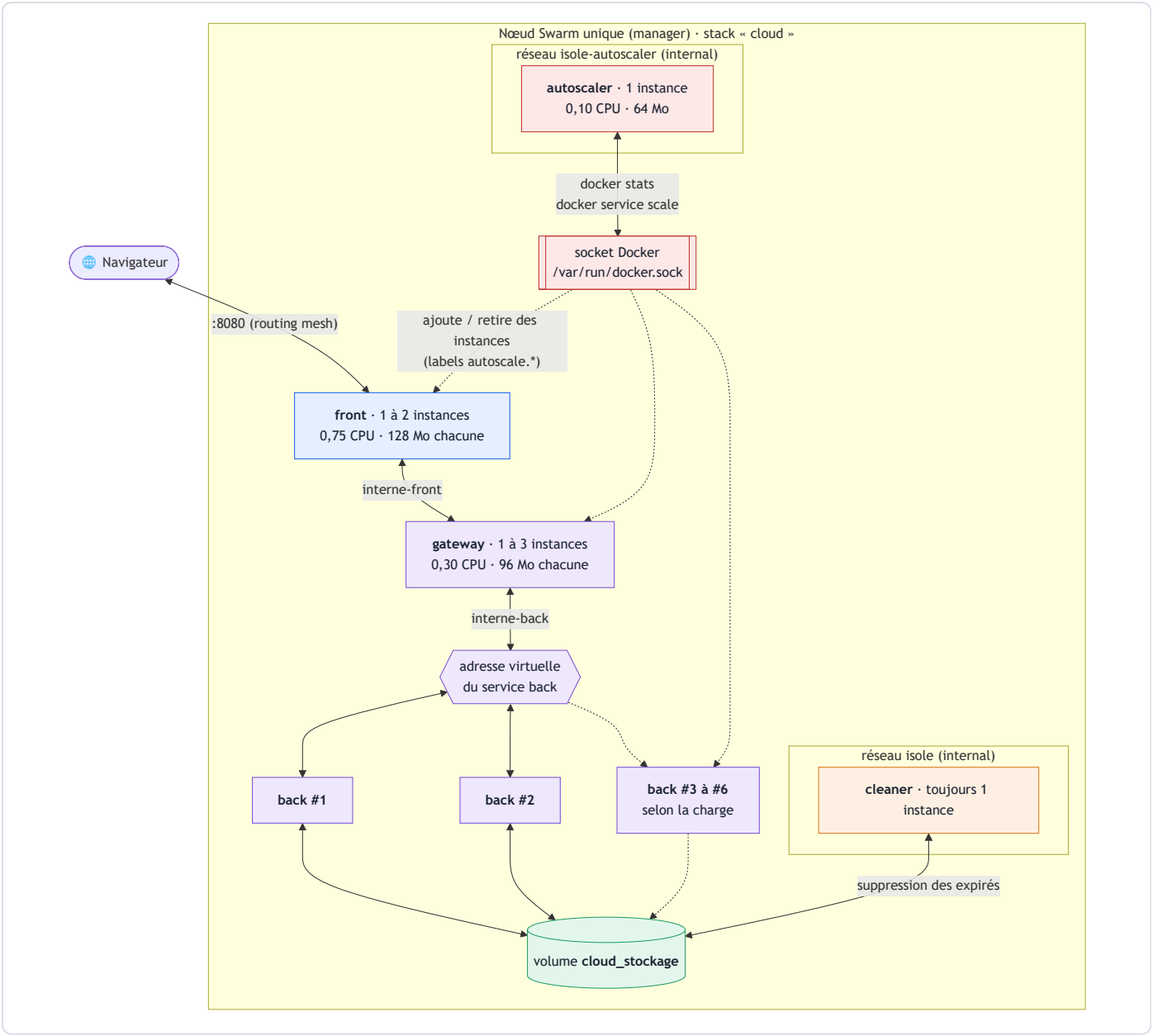
Service	Mécanisme	Arrêt	Code
front, back	`tini` relaie SIGTERM, Node ferme le serveur	0,73 s / 0,47 s	**0**
gateway	`STOPSIGNAL SIGQUIT` : nginx finit les requêtes en cours	0,53 s	**0**
cleaner	`trap` ; suppression en arrière-plan + `wait` , groupe `setsid` arrêté d'un coup ; chaque suppression est atomique	0,5 à 0,6 s en pleine passe (10 000 fichiers)	**0**
autoscaler	`trap` ; relevé et `sleep` en arrière-plan + `wait`	0,6 s	**0**

`0` = signal traité ; `137` = signal ignoré, tué au bout de `stop_grace_period: 10s` . Vérifié avec `docker inspect -f '{{.State.ExitCode}}'` .

Scalabilité : Swarm et autoscaling

Schéma complet

`docker-stack.yml` : mêmes images et même `.env` ; le nombre d'instances suit la charge grâce à notre autoscaler.



Public **Interne** (overlay `internal`) **Données** **Isolé** (réseau sans autre membre)

Pilotage (moteur Docker) ━━ échange permanent ┈┈ variable : instances ajoutées sous charge, ou action par le socket

Règle de l'autoscaler	Valeur	Pourquoi
Monter (+1)	CPU moyen > 70 % de la limite, ou mémoire > 80 %, 3 relevés de suite	Une instance met 10 à 15 s à être prête : on anticipe pour rester sous 80 %
Descendre (−1)	CPU < 25 % et mémoire < 50 %, 6 relevés	Hystérésis : sinon le service oscillerait
Bornes	back 2–6 · gateway 1–3 · front 1–2 · cleaner 1	Au-delà, aucun gain mesuré ; 2 back pour la disponibilité
Budget machine	Somme des limites ≤ 80 % du CPU et de la RAM de l'hôte	La consigne « jamais plus de 80 % », appliquée à toute la stack
Socket Docker	Seul montage, non-root, lecture seule, aucun réseau	Le socket vaut un accès root : risque assumé et réduit

Résultats mesurés et Docker face aux VM Objectif pédagogique

Ce qui a été vérifié

Test	Résultat
Démarrage `docker compose up -d --build --wait`	4 services *healthy*, ordre back → gateway / cleaner → front
Réseau : qui peut joindre qui	Front → back **bloqué** ; back et gateway sans Internet ; cleaner sans interface réseau
Cycle de vie d'un fichier (durée 1 min)	`200` → expiré : `410` → après la passe du cleaner : supprimé, `404`
Charge (5 000 requêtes, 50 en parallèle)	Compose : 166 req/s · Swarm 3 back : 431 à 574 req/s · **0 échec**
Autoscaling (3 min de charge)	back 2 → 3 en 41 s, gateway 1 → 2 en 92 s ; retour aux minimums 72 s après ; **0 échec** sur 94 000 requêtes
Budget machine abaissé à 30 %	Montées refusées et journalisées
Auto-réparation (instance tuée)	Recréée en 13 s, **0 erreur** sur 179 requêtes
Mise à jour progressive	**0 erreur** sur 520 requêtes
Arrêt de chaque service	Moins d'1 s, code **0**

Docker face aux machines virtuelles (mesuré sur cette stack)

	Machines virtuelles	Conteneurs Docker
Ajouter une instance	Une VM complète : minutes, plusieurs Go	2 instances de plus en 17 s, ~27 Mo de RAM chacune
Instance en panne	Intervention ou outillage dédié	Relancée seule en 13 s
Mise à jour	Arrêt de service ou bascule manuelle	Remplacement progressif, 0 erreur
Empreinte	Un OS complet par VM	~100 Mo de RAM au repos pour toute l'application
Configuration	À reproduire sur chaque VM	Identique partout : même image, même `.env`

Limites connues

Un seul nœud Swarm	Plusieurs machines demanderaient un registre (`registry:2` vient du Hub) et un stockage partagé
Rafale d'envois	Plus rapide que l'autoscaler (~15 s contre ~40 s) : protégée à la source par des tampons TCP plafonnés (`sysctls`) ; ~23 envois de 40 Mo simultanés sous 80 % par front (avant : OOM dès 20)

CI/CD : `ci/pipeline.sh` (ou `.ps1`), chaque étape dans son conteneur alpine : build → lint ‖ tests de l'API → audit npm ‖ règles des images → tests de bout en bout sur une stack isolée → déploiement Swarm progressif avec retour arrière automatique. Pipeline complet : 153 s ; 0 échec sur 86 332 requêtes pendant le déploiement (détail : `docs/ci-cd.md`).

Rejouer les tests : outil de charge `Bench/` (alpine + `ab`) ; démonstration de l'autoscaling en une commande : `./Bench/run-autoscale-demo.sh` (mode d'emploi : `docs/tester-la-charge.md`).

Détail des mesures, des tests et de chaque choix : dossier `docs/` du dépôt · github.com/AbadieThomasPro/M2_TP_Docker